

Documentação Técnica da Pipeline

Classificador de Doenças do Cafeeiro

Pipeline completa, da preparação dos dados e do treinamento do modelo até o deploy da API de inferência na nuvem, com os problemas encontrados, as soluções aplicadas e as saídas reais registradas. Versão corrigida em 02/10/2026.

1. Visão geral da arquitetura

A pipeline vai da ingestão das imagens do dataset DigiPathos até um endpoint REST de inferência hospedado no Render. O fluxo é sequencial:

1. Dataset DigiPathos	2. Pré-processamento e augmentation	3. Transfer learning MobileNetV2	4. Exportação modelo_cafe.keras	5. API FastAPI + Uvicorn	6. Deploy no Render
--------------------------	---	-------------------------------------	------------------------------------	-----------------------------	------------------------

2. Dados e treinamento do modelo

2.1 Dataset

- **Fonte:** DigiPathos (Embrapa), publicado no Redape: doi:10.48432/XA1OVL (<https://www.redape.dados.embrapa.br/dataset.xhtml?persistentId=doi:10.48432/XA1OVL>).
- **Licença e citação:** consulte a página do dataset e cite a fonte em qualquer aplicação que use o modelo.
- **Divisão dos dados:** 80% treino e 20% validação, com seed=42 nos dois subconjuntos para que não se sobreponham.

2.2 Classes (6 categorias)

As classes vêm das subpastas do dataset, em ordem alfabética (comportamento do `image_dataset_from_directory`). A lista `CLASSES` do `main.py` precisa ter exatamente esta ordem; se divergir, os nomes saíem trocados.

Índice	Classe (saída da API)
0	bicho_mineiro
1	cercosporiose
2	ferrugem
3	mancha_aureolada
4	mancha_mantegosa
5	phoma

Atenção: o modelo **não tem classe "saudável"**. Toda imagem recebe uma das seis doenças, mesmo uma folha sadia ou uma imagem que não seja de folha de café.

2.3 Pré-processamento e aumento de dados

- **Tamanho:** imagens padronizadas em 224 x 224 x 3 (RGB), lotes de 32.
- **Augmentation (somente no treino):** RandomFlip (horizontal e vertical), RandomRotation e RandomZoom.
- **Normalização:** escalonamento dos pixels para o intervalo [-1, 1], exigido pela MobileNetV2. Essa etapa está **dentro do modelo** (camada de pré-processamento antes da rede base). Por isso a API envia os pixels em 0–255, **sem normalizar**. Quem reimplementar a API e normalizar de novo vai degradar as previsões.

2.4 Arquitetura (transfer learning)

- **Rede base:** MobileNetV2 com pesos do ImageNet e camadas convolucionais congeladas.
- **Cabeçote de classificação:** GlobalAveragePooling2D, Dropout e Dense(6, activation='softmax').

- **Compilação:** otimizador Adam e perda categorical_crossentropy.

2.5 Serialização

- **Arquivo:** modelo_cafe.keras (arquitetura + pesos), salvo em 02/10/2026.
- **Versão do Keras que salvou o modelo: 3.15.1** (registrada no metadata.json dentro do arquivo). Esse número determina as dependências do servidor (seção 4.3).

2.6 Avaliação

A avaliação completa (acurácia de validação, matriz de confusão e métricas por classe) é gerada por script_de_avalia_o_de_m_tricas_e_matriz_de_confus_o.py, com as figuras em figuras.zip, ambos no repositório. Os valores obtidos devem ser consultados nessas saídas; um único teste (seção 6) não substitui a validação.

3. Serviço web (FastAPI)

3.1 Endpoints

Método e rota	Função	Resposta
GET /	Verifica se o serviço está no ar.	{"status": "online", "modelo": "MobileNetV2 DigiPathos"}
POST /predict	Recebe uma imagem em multipart/form-data, campo file	JSON com diagnóstico, confiança e probabilidades.
GET /docs	Documentação interativa (Swagger UI), gerada automaticamente pelo FastAPI.	Página HTML.

3.2 Comportamento do /predict

- **Formatos aceitos:** JPEG e PNG. Outro tipo retorna 400 ("Formato de imagem inválido"). Campo file ausente retorna 422.
- **Processamento:** a imagem é lida em memória, convertida para RGB e redimensionada para 224 x 224, sem normalização (ver 2.3).
- **Resposta:** diagnostico (classe de maior probabilidade), confianca (0 a 100, em %) e probabilidades (as seis classes, em %, somando 100).
- **CORS:** o CORSMiddleware libera GET e POST de qualquer origem (allow_origins=["*"]), necessário para chamadas feitas por navegador. Em produção, restrinja ao domínio do aplicativo.
- **Privacidade:** o código não grava a imagem em disco nem em log; ela existe só em memória durante a requisição, e o redimensionamento descarta os metadados EXIF (inclusive GPS).

3.3 Código da aplicação (main.py)

```
import io
import numpy as np
import tensorflow as tf
from fastapi import FastAPI, File, UploadFile, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image

app = FastAPI(title="API Doenças do Café")

# Libera chamadas vindas do app (necessário para o navegador)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)

CLASSES = [
    'bicho_mineiro',
    'cercosporiose',
    'ferrugem',
    'mancha_aureolada',
    'mancha_mantegosa',
    'phoma'
]

# Carrega o modelo apenas uma vez
MODEL_PATH = "modelo_cafe.keras"
model = tf.keras.models.load_model(MODEL_PATH, compile=False)

@app.get("/")
def root():
    return {"status": "online", "modelo": "MobileNetV2 DigiPathos"}

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    if file.content_type not in ["image/jpeg", "image/png", "image/jpg"]:
        raise HTTPException(status_code=400, detail="Formato de imagem inválido.")

    contents = await file.read()
    image = Image.open(io.BytesIO(contents)).convert("RGB").resize((224, 224))
    img_array = np.array(image, dtype=np.float32)
    img_batch = np.expand_dims(img_array, axis=0)

    predictions = model.predict(img_batch, verbose=0)[0]
    top_index = int(np.argmax(predictions))

    return {
        "diagnostico": CLASSES[top_index],
        "confianca": round(float(predictions[top_index]) * 100, 2),
        "probabilidades": {CLASSES[i]: round(float(predictions[i]) * 100, 2) for i in range(len(CLASSES))}
    }
```

4. Dependências, runtime e deploy

4.1 Controle de versão

- **Repositório:** github.com/Ralha-igor/FastAPI-ML, branch main.
- **Autenticação:** o GitHub não aceita mais senha em operações Git. O push por HTTPS usa um Personal Access Token (PAT) no lugar da senha.

4.2 Configuração no Render

Item	Valor
Tipo de serviço	Web Service (plano gratuito)
Python	3.11.9 (arquivo .python-version e variável de ambiente PYTHON_VERSION=3.11.9)
Build command	pip install -r requirements.txt
Start command	uvicorn main:app --host 0.0.0.0 --port \$PORT

Cuidado: a variável de ambiente `PYTHON_VERSION` tem prioridade sobre o `.python-version`. Os dois precisam ter o mesmo valor, escrito sem zero à esquerda.

4.3 requirements.txt

```
fastapi==0.110.0
uvicorn==0.28.0
tensorflow-cpu==2.18.0
keras==3.15.1
pillow==10.2.0
python-multipart==0.0.9
numpy==1.26.4
```

Regra de compatibilidade: o Keras do servidor precisa ser **igual ou mais novo** que o Keras que salvou o modelo (3.15.1). O Keras 3.15.1 exige **Python 3.11 ou superior**; por isso o Python 3.10.12 e o TensorFlow 2.16.1, usados nas primeiras tentativas, não funcionam. Ao retreinar, anote a versão do Keras e atualize este arquivo.

5. Problemas encontrados e soluções

Problema	Causa	Solução
Falha ao carregar o modelo no Render: <code>GlorotUniform.__init__()</code> got an unexpected keyword argument 'input_axes'	O modelo foi salvo com Keras 3.15.1, mas o servidor instalava um Keras 3 mais antigo, que não conhece os campos <code>input_axes</code> e <code>output_axes</code> . Trocar apenas para o TensorFlow 2.16.1 não resolveu.	Python 3.11.9, <code>tensorflow-cpu==2.18.0</code> e <code>keras==3.15.1</code> ; build com Clear build cache & deploy.
Build falha: Unable to install Python version 3.11.09	A variável <code>PYTHON_VERSION</code> estava com um zero a mais (3.11.09) e prevalece sobre o <code>.python-version</code> .	Corrigir a variável para 3.11.9 no painel do Render (Environment).
Aviso gzip: invalid compressed data--crc error no início do build	Cache de build corrompido.	Inofensivo (o Render segue sem cache). Clear build cache & deploy remove o aviso.
Push recusado: Password authentication is not supported	O GitHub não aceita senha para operações Git.	Gerar um Personal Access Token e usá-lo como senha.
Chamada do aplicativo bloqueada pelo navegador	Faltava CORS na API.	Adicionar o <code>CORSMiddleware</code> no <code>main.py</code> (seção 3.3).

6. Saídas registradas do processo

6.1 Erro de carregamento do modelo (versão do Keras incompatível)

```
File "/opt/render/project/src/main.py", line 20, in <module>
    model = tf.keras.models.load_model(MODEL_PATH, compile=False)
...
TypeError: <class 'keras.src.models.functional.Functional'> could not be deserialized properly.
...
Exception encountered: GlorotUniform.__init__() got an unexpected keyword argument 'input_axes'
==> Exited with status 1
```

6.2 Erro de build causado pela variável `PYTHON_VERSION`

```
==> Using Python version 3.11.09 via environment variable PYTHON_VERSION
==> Installing Python version 3.11.09...
==> Attempting to build Python 3.11.09 from source...
==> Unable to install Python version 3.11.09
```

6.3 Inicialização bem-sucedida (02/10/2026)

```
INFO: Started server process [57]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:10000 (Press CTRL+C to quit)
==> Your service is live
```

6.4 Teste em produção

Requisição GET / retornou {"status": "online", "modelo": "MobileNetV2 DigiPathos"}. Em seguida, a imagem phoma3.jpeg foi enviada pelo Swagger UI para POST /predict, com resposta 200 OK:

Classe	Probabilidade (%)
phoma (diagnóstico)	88,73
cercosporiose	8,23
mancha_aureolada	1,49
bicho_mineiro	1,34
ferrugem	0,20
mancha_mantegosa	0,01

A classe prevista coincide com a doença da imagem. É um teste de fumaça: confirma que o modelo carrega, que o pré-processamento está coerente e que o formato da resposta está correto. Não mede a qualidade do modelo.

7. Como usar a API

Endereço do endpoint: POST <https://fastapi-ml-7fcy.onrender.com/predict>, imagem no campo file.

cURL

```
curl -X POST "https://fastapi-ml-7fcy.onrender.com/predict" \
-H "accept: application/json" \
-F "file=@/caminho/para/folha_cafe.jpg;type=image/jpeg"
```

JavaScript (navegador)

```
const form = new FormData();
form.append("file", arquivo); // File vindo de <input type="file">

const res = await fetch("https://fastapi-ml-7fcy.onrender.com/predict", {
  method: "POST",
  body: form, // não defina o Content-Type manualmente
});
const dados = await res.json();
console.log(dados.diagnostico, dados.confianca, dados.proBABilidades);
```

8. Limitações e cuidados

- **Sem classe "saudável":** o modelo sempre aponta uma doença. Folhas saudáveis e imagens que não são de café também recebem diagnóstico. Uma solução é treinar com uma classe saudável ou rejeitar previsões abaixo de um limiar de confiança.
- **Confiança não é certeza:** o valor vem do softmax e pode ser alto mesmo quando o modelo erra. O resultado deve ser apresentado como hipótese, com confirmação de um técnico agrônomo.
- **Doenças parecidas:** como no teste da seção 6.4, cercosporiose e phoma ficam próximas em probabilidade; avalie a matriz de confusão antes de confiar numa classe isolada.
- **Plano gratuito do Render:** o serviço hiberna por inatividade e a primeira requisição pode levar cerca de 50 segundos. Use timeout de pelo menos 90 s no cliente e mostre um indicador de carregamento.
- **CORS aberto:** allow_origins=["*"] serve para desenvolvimento e demonstração. Restrinja ao domínio do app em produção.

- **Privacidade:** o app que envia a foto a esta API deve informar que a imagem é enviada para análise e não é armazenada. Não afirme que ela “não sai do celular” enquanto a inferência for remota.
- **Versionamento:** ao salvar um novo modelo, registre a versão do Keras e mantenha o `requirements.txt` igual ou mais novo.

9. URLs do serviço

Recurso	Endereço
API (status)	https://fastapi-ml-7fcy.onrender.com
Documentação interativa (Swagger UI)	https://fastapi-ml-7fcy.onrender.com/docs
Endpoint de predição (POST)	https://fastapi-ml-7fcy.onrender.com/predict
Repositório	https://github.com/Ralha-igor/FastAPI-ML
Dataset DigiPathos (Embrapa)	https://www.redape.dados.embrapa.br/dataset.xhtml?persistentId=doi:10.48432/XA1OVL